

5주차

실습에 대한 설명은 .ipynb에 있습니다. 일부 주석은 여기에도 정리했습니다.

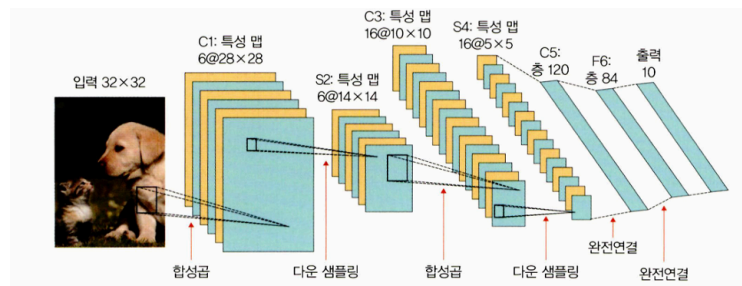
[딥러닝 파이토치 교과서] 6장

6.1 이미지 분류를 위한 신경망

입력 데이터로 이미지를 사용한 분류는 특정 대상이 영상 내에 존재하는지 여부 판단함

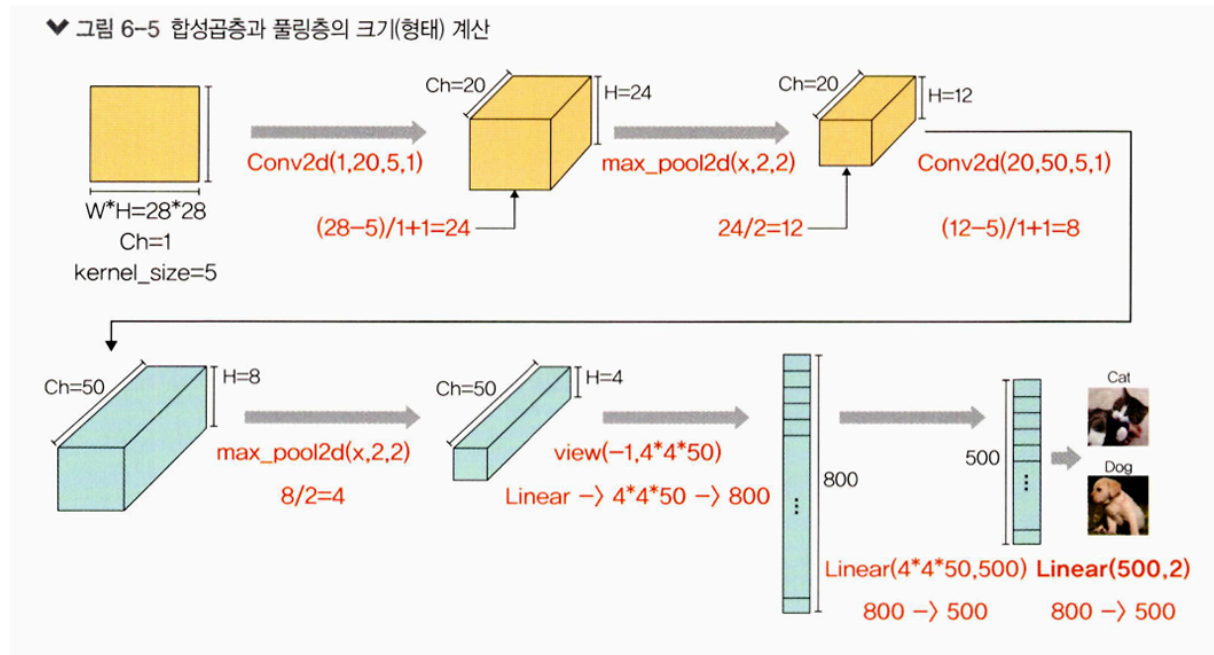
6.1.1 LeNet-5

- 합성곱 신경망이라는 개념을 최초로 얀 르쿤이 개발한 구조
- 현재 CNN의 초석
- 합성곱(convolutional)과 다운 샘플링(sub-sampling(or pooling))을 반복 $\Rightarrow$ 마지막에 완전 연결층에서 분류 수행



1. C1에서 5x5 합성곱 연산 후 28x28크기의 feature map 6개 생성
 2. S2에서 다운 샘플링 $\rightarrow$ feature map 크기 14x14로 축소
 3. C3에서 5x5 합성곱 연산 $\rightarrow$ 10x10 feature map 16개 생성
 4. S4에서 다운 샘플링 $\rightarrow$ feature map 크기 5x5로 줄임
 5. C5에서 5x5 합성곱 연산 $\rightarrow$ 1x1 feature map 120개 생성
 6. F6에서 완전연결층으로 C5결과를 unit 84개에 연결시킴
- $\Rightarrow$ C: 합성곱층, S: 풀링층, F: 완전연결층

합성곱층과 풀링층의 크기(형태) 계산



```
from torchvision import transforms #이미지 변환(전처리) 기능을
제공하는 라이브러리
from torch import optim #경사 하강법을 이용 -> 가중치를 구하기
위한 옵티마이저 라이브러리
import os # 파일경로에 대한 함수 제공
from tqdm import tqdm_notebook as tqdm
# 진행 상황을 가시적으로 표현. 모델의 학습 경과를 확인하고 싶을 때
사용
```

```
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
# 파이토치는 텐서플로우와 달리 GPU를 자동으로 할당 안해줌
```

```
'train': transforms.Compose([ # 이미지 변형할 수 있는 방식들의
목록
    transforms.RandomResizedCrop(resize, scale=
(0.5, 1.0)), # 입력 이미지를 주어진 크기로 조정
    transforms.RandomHorizontalFlip(), # 주어진
```

확률로 이미지를 수평 반전시킴

```
transforms.ToTensor(),  
# ImageFolder메소드는 이미지를 읽을 때 PIL 사용  
함. 이때, 효율적인 연산을 위해 torch.FloatTensor 배열로 바꿔야 함  
transforms.Normalize(mean, std)  
# train_dataset을 사용하려면 정규화 해야함  
]),
```

random.seed(num): 난수 생성

sorted: 데이터를 정렬된 리스트로 만들어 반환

모멘텀 SGD

- SGD에 관성 추가한 것.
- 매번 기울기를 구하지만, 가중치를 수정하기 전에 이전 수정방향을 참고
⇒ 같은 방향으로 일정한 비율만 수정되게 함

Conv2d 계층에서의 출력 크기 구하는 공식

출력 크기 = $(W - F + 2P) / S + 1$

- W: input_volume_size
- F: kernel_size
- P: padding_size
- S: strides

MaxPool2d 계층에서의 출력 크기 구하는 공식

출력 크기 = IF / F

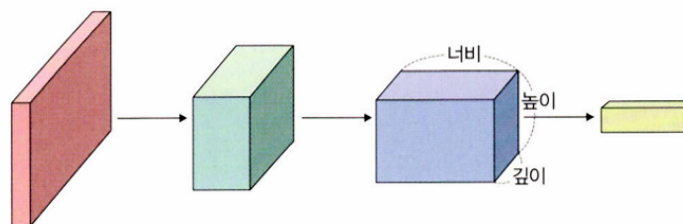
- IF: input_filter_size
- F: kernel_size

6.1.2 AlexNet

- ImageNet 영상 DB 기반으로 한 화상 인식 대회에서 우승한 CNN구조
- ReLU 함수에 대한 inplace 연산: 연산에 대한 결과값을 새로운 변수에 저장하지 않
기존 데이터를 대체하는 것 ⇒ 기존 값들 무시
- AlexNet은 깊이가 깊은 네트워크를 사용함. 그렇지 않으면 풀링층 때문에 크기 감소로 오류 발생할 수 있음

CNN 구조

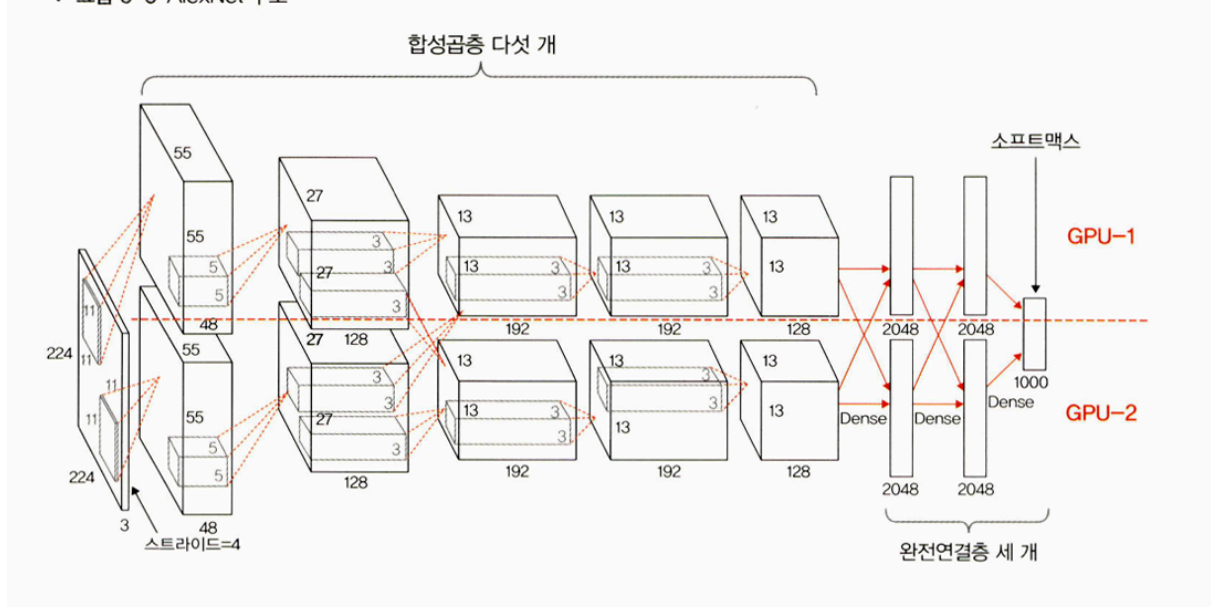
▼ 그림 6-8 CNN 구조



- 3차원(너비, 높이, 깊이)
- 색상 많은 이미지는 R, G, B ⇒ 시작 = 3
- 합성곱을 거침 → feature map 생성 → 중간 영상의 깊이가 달라짐

AlexNet 구조

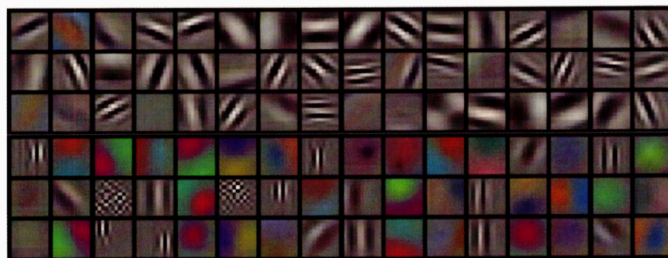
▼ 그림 6-9 AlexNet 구조



▼ 표 6-2 AlexNet 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

- 합성곱층 5개 + 완전 연결층 3개
- 마지막 완전연결층은 softmax함수 사용함
- GPU 2개를 기반으로한 병렬 구조
- 첫번째 합성곱층 커널 크기 = 11x11x3
- 스트라이드 = 4
- GPU-1: 컬러와 상관없는 정보 추출하기 위한 커널 학습
- GPU-2: 컬러와 관련된 정보를 추출하기 위한 커널 학습
- AlexNet GPU-1, 2 적용 결과



AvgPool2d

- 풀링을 위해 사용
- 풀링에 대한 커널, 스트라이드 크기, 패딩을 정의해야 동작함

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel_size}[0]}{\text{stride}[0]} + 1 \right\rfloor$$
$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel_size}[1]}{\text{stride}[1]} + 1 \right\rfloor$$

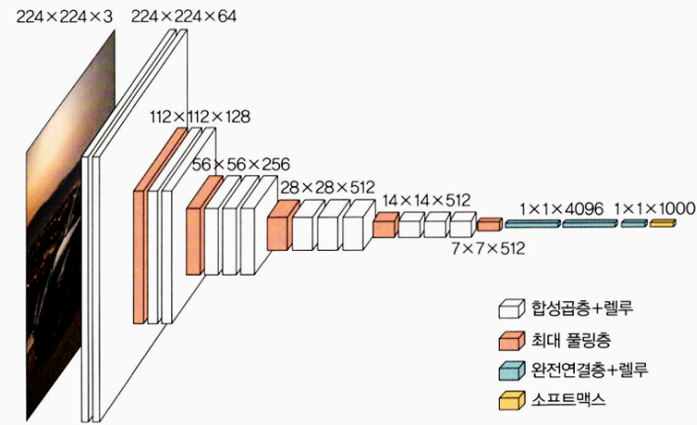
AdaptiveAvgPool2d

- 풀링 작업이 끝날 때 필요한 출력 크기를 정의함
- 출력 크기 조정이 쉬워짐
- 입력 크기에 변동있고 CNN 위쪽에 완전연결층을 사용할 때 유용함

6.1.3 VGGNet

- 목적: 합성곱층의 파라미터 수 줄이고 훈련시간 개선
네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인
- 연구 팀은 깊이의 영향만 최대한 확인하기 위해 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3x3으로 고정함
- 구조

♥ 그림 6-13 VGG16 구조



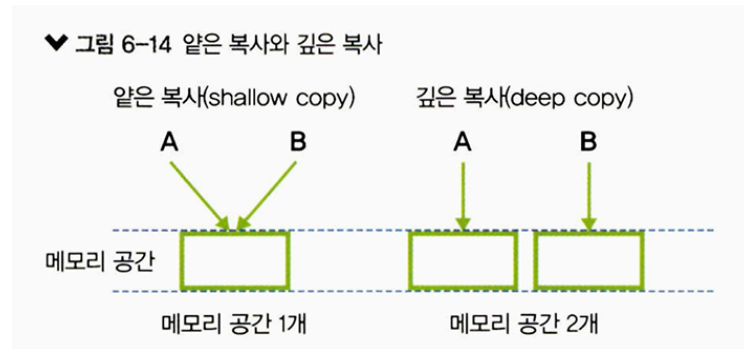
♥ 표 6-3 VGG16 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—

○ 계속

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

객체 복사



- 단순 복사: 복사한 값과 원본 값 모두 바뀜
- 얕은 복사: `copy.copy()` , 복사값만 바뀜
- 깊은 복사: `copy.deepcopy()` , 복사값이 바뀌지만 메모리 수가 지시하는 변수의 수와 같

isinstance

- 조건이 T/F인지 확인함

```
print(isinstance(1,int)) #1이 int인지 판단
print(isinstance(1.2,int)) #1.2가 int인지 판단
print(isinstance('deep learning',str)) #deep learning이
str인지 판단
```

- max: 텐서의 요소 중 가장 높은 값 반환
- argmax: 텐서의 요소 중 가장 높은 값의 인덱스 반환
- keepdim = True: 출력 텐서를 입력과 동일한 크기로 유지
- zip(): 여러 개의 리스트(튜플)를 합침 → 새로운 튜플 타입 반환
- torch.cat: 텐서 연결할 때 사용
- torch.clamp: min, max 범주에 이미지가 위치하도록 함

람다 함수

♥ 그림 6-19 일반 함수와 람다 함수

```
def 함수명(매개변수):  
    return 반환될 결과값  
  
lamda 매개변수 : 반환될 결과값
```

- 정의와 동시에 사용 가능
- 함수명 X, 저장된 변수 X ⇒ 재사용 불가능

torch.add

- 새로운 메모리 공간이 할당되어 저장됨

```
x = torch.tensor([1,2])  
y = x.add(10) # torch.add 적용  
print(y) # tensor([11,12])  
print(x is y) # FALSE
```

torch.add_

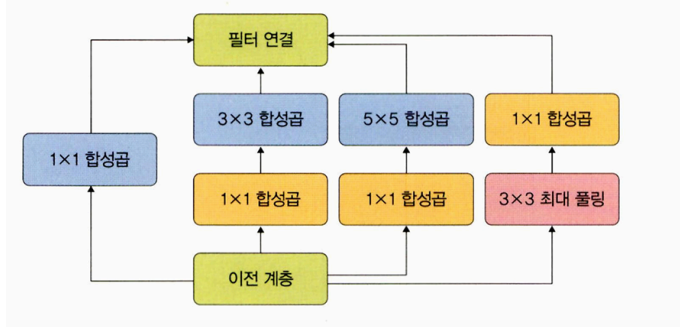
- 새로운 공간 할당 없이 기존 메모리에 위치한 값을 대체

```
x = torch.tensor([1,2])  
y = x.add_(10) # torch.add_ 적용  
print(y) # tensor([11,12])  
print(x is y) # True
```

6.1.4 GoogLeNet

- 주어진 HW 자원을 최대한 효율적으로 이용
- 학습 능력 극대화 가능
- 깊고 넓은 신경망
- inception 모듈이 추가됨

♥ 그림 6-23 GoogLeNet의 인셉션 모듈



- 특징을 효율적으로 추출하기 위해 1x1, 3x3, 5x5 합성곱 연산 각각 수행
- 3x3 최대 풀링은 입출력의 높이, 너비가 같아야 함 ⇒ 패딩 추가 필요
- 연산
 - 1x1 합성곱
 - 1x1 합성곱 + 3x3 합성곱
 - 1x1 합성곱 + 5x5 합성곱
 - 3x3 최대풀링 + 1x1 합성
- 희소 연결: correlation이 높은 노드끼리만 연결하는 방법
연산량 감소 ⇒ overfitting 해결